AG3 - Actividad Guiada 3

Nombre: Miguel Laversa

En esta actividad comparo búsqueda aleatoria, búsqueda local y recocido simulado sobre la instancia `swiss42` del problema del viajante. La instancia queda incluida en el notebook para que la ejecución sea reproducible y no dependa de una descarga externa.

1. Preparación del problema

Una solución es una permutación de las 42 ciudades. La función objetivo suma todas las aristas del recorrido, incluida la vuelta a la ciudad inicial.

Los datos corresponden a la instancia `swiss42` de TSPLIB.

```
Instancia: swiss42
Número de ciudades: 42
```

In [2]:
```python
def crear_solucion(nodos, generador):
    # Fijar la primera ciudad elimina rutas equivalentes obtenidas por rotación.
    resto = nodos[1:]
    generador.shuffle(resto)
    return [nodos[0], *resto]

def distancia_total(solucion, distancias=DISTANCIAS):
    # El último término cierra el circuito al volver a la ciudad inicial.
    return sum(
        distancias[solucion[i], solucion[(i + 1) % len(solucion)]]
        for i in range(len(solucion))
    )

def coordenadas_mds(distancias):
    # MDS produce coordenadas solo para visualizar la matriz de distancias.
    n = len(distancias)
    centrado = np.eye(n) - np.ones((n, n)) / n
    matriz_gram = -0.5 * centrado @ (distancias ** 2) @ centrado
    valores, vectores = np.linalg.eigh(matriz_gram)
    indices = np.argsort(valores)[-2:]
    return vectores[:, indices] * np.sqrt(np.maximum(valores[indices], 0))

COORDENADAS = coordenadas_mds(DISTANCIAS)

def dibujar_ruta(solucion, titulo):
    ruta = solucion + [solucion[0]]
    puntos = COORDENADAS[ruta]
    plt.figure(figsize=(7, 5))
    plt.plot(puntos[:, 0], puntos[:, 1], "o-", markersize=4, linewidth=1)
    for nodo, (x, y) in enumerate(COORDENADAS):
        plt.text(x, y, str(nodo), fontsize=7)
    plt.title(f"{titulo}\nDistancia: {distancia_total(solucion):.0f}")
    plt.axis("equal")
    plt.axis("off")
    plt.show()
```

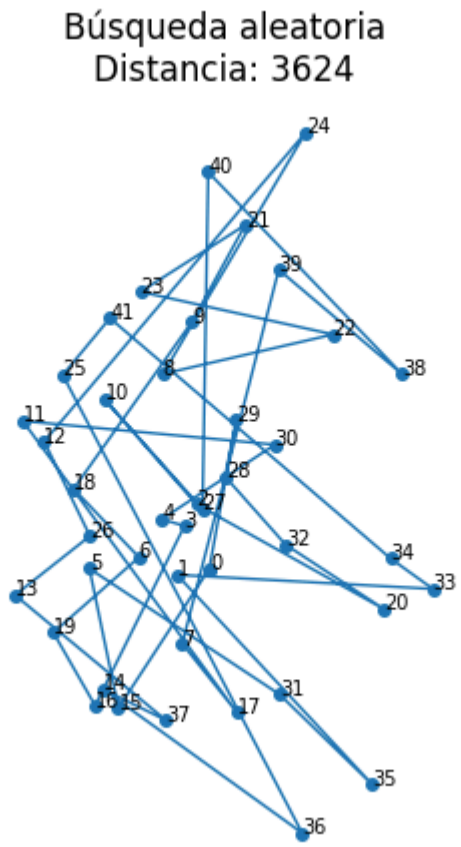
2. Búsqueda aleatoria

Se generan soluciones independientes y se conserva la mejor. El método explora distintas zonas, pero no utiliza la información obtenida en iteraciones anteriores.

In [3]:
```
def busqueda_aleatoria(iteraciones, semilla=0):
    generador = random.Random(semilla)
    mejor = None
    mejor_distancia = math.inf
    for _ in range(iteraciones):
        candidata = crear_solucion(NODOS, generador)
        distancia = distancia_total(candidata)
        if distancia < mejor_distancia:
            mejor, mejor_distancia = candidata, distancia
    return mejor, mejor_distancia

solucion_aleatoria, distancia_aleatoria = busqueda_aleatoria(10_000,
semilla=42)
print("Mejor distancia:", distancia_aleatoria)
dibujar_ruta(solucion_aleatoria, "Búsqueda aleatoria")
```

```
Mejor distancia: 3624.0
```



3. Búsqueda local con vecindad 2-opt

Un movimiento 2-opt invierte un segmento de la ruta. En cada iteración se acepta la primera mejora encontrada y el proceso termina cuando una exploración completa no produce mejoras.

In [4]:
```
def movimiento_2opt(solucion, i, j):
    return solucion[:i] + list(reversed(solucion[i:j + 1])) + solucion[j + 1:]

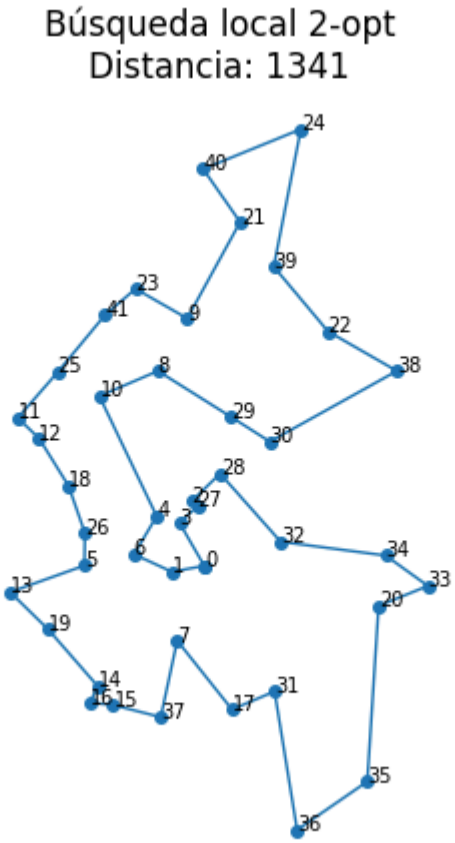
def busqueda_local(semilla=0, solucion_inicial=None):
    generador = random.Random(semilla)
    actual = solucion_inicial[:] if solucion_inicial else crear_solucion(NODOS,
generador)
    distancia_actual = distancia_total(actual)
```

```
    evaluaciones = 0

    while True:
        mejoro = False
        for i in range(1, len(actual) - 1):
            for j in range(i + 1, len(actual)):
                candidata = movimiento_2opt(actual, i, j)
                distancia = distancia_total(candidata)
                evaluaciones += 1
                if distancia < distancia_actual:
                    # Estrategia de primera mejora: se reinicia la exploración
al aceptar.
                    actual, distancia_actual = candidata, distancia
                    mejoro = True
                    break
            if mejoro:
                break
        if not mejoro:
            return actual, distancia_actual, evaluaciones

solucion_local, distancia_local, evaluaciones_local =
busqueda_local(semilla=42)
print("Mejor distancia:", distancia_local)
print("Soluciones vecinas evaluadas:", evaluaciones_local)
dibujar_ruta(solucion_local, "Búsqueda local 2-opt")
```

```
Mejor distancia: 1341.0
Soluciones vecinas evaluadas: 22035
```



4. Recocido simulado

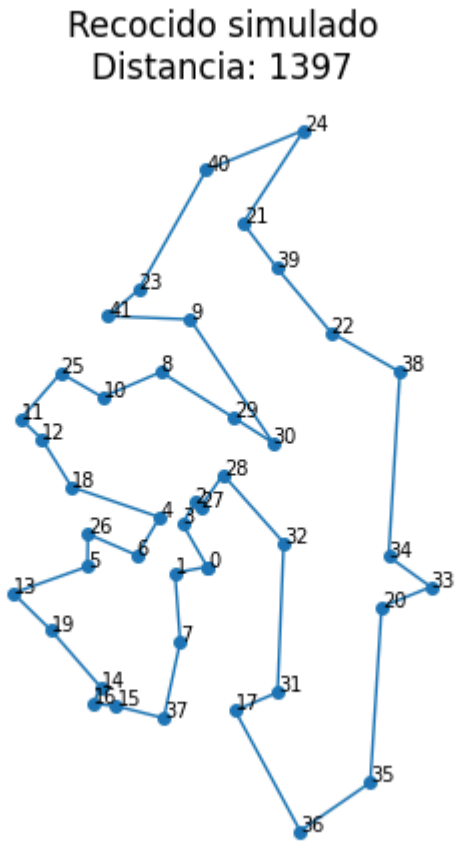
Las mejoras se aceptan siempre. Un empeoramiento de magnitud Δ se acepta con probabilidad $e^{-\Delta/T}$. Al reducir la temperatura disminuye gradualmente la posibilidad de abandonar la región actual.

In [5]:
```python
def recocido_simulado(
    temperatura_inicial=200,
    enfriamiento=0.9995,
    iteraciones=25_000,
    semilla=0,
):
    generador = random.Random(semilla)
    actual = crear_solucion(NODOS, generador)
    distancia_actual = distancia_total(actual)
    mejor = actual[:]
    mejor_distancia = distancia_actual
    temperatura = temperatura_inicial

    for _ in range(iteraciones):
        i, j = sorted(generador.sample(range(1, len(actual)), 2))
        candidata = movimiento_2opt(actual, i, j)
        distancia = distancia_total(candidata)
        delta = distancia - distancia_actual
        # La aceptación probabilística permite escapar temporalmente de un
mínimo local.
        if delta < 0 or generador.random() < math.exp(-delta / max(temperatura,
1e-12)):
            actual, distancia_actual = candidata, distancia
        if distancia_actual < mejor_distancia:
            mejor, mejor_distancia = actual[:], distancia_actual
        temperatura *= enfriamiento
    return mejor, mejor_distancia

solucion_recocido, distancia_recocido = recocido_simulado(semilla=42)
print("Mejor distancia:", distancia_recocido)
dibujar_ruta(solucion_recocido, "Recocido simulado")
```

```
Mejor distancia: 1397.0
```



5. Ejercicio adicional - Comparación experimental

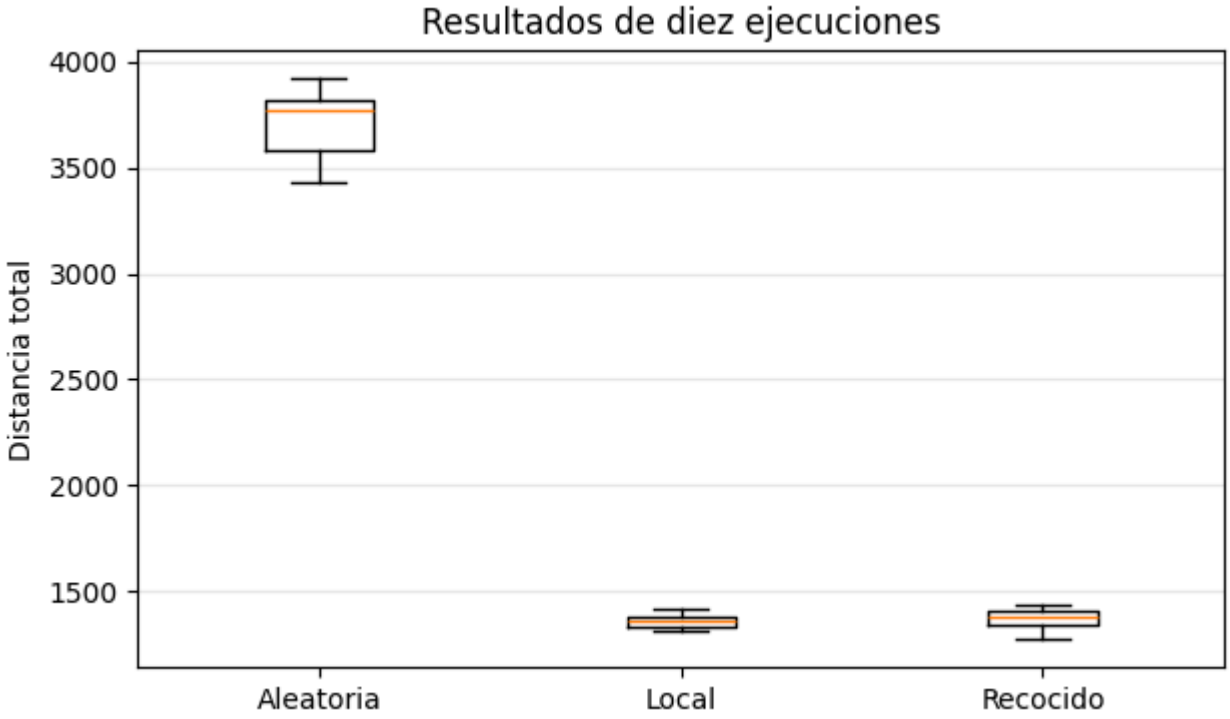
Como los métodos incluyen aleatoriedad, una sola ejecución no permite compararlos con suficiente información. Repito cada método con diez semillas y calculo mínimo, media y desviación estándar.

In [6]:
```
# Las mismas semillas reducen la variación ajena al método comparado.
resultados = {"Aleatoria": [], "Local": [], "Recocido": []}
for semilla in range(10):
    resultados["Aleatoria"].append(busqueda_aleatoria(3_000, semilla)[1])
    resultados["Local"].append(busqueda_local(semilla)[1])
    resultados["Recocido"].append(
        recocido_simulado(iteraciones=12_000, semilla=semilla)[1]
    )

print(f"{'Método':<12} {'Mínimo':>10} {'Media':>10} {'Desv.':>10}")
for metodo, valores in resultados.items():
    print(f"{metodo:<12} {min(valores):>10.1f} {mean(valores):>10.1f}
{pstdev(valores):>10.1f}")

plt.figure(figsize=(7, 4))
plt.boxplot(resultados.values(), tick_labels=resultados.keys())
plt.ylabel("Distancia total")
plt.title("Resultados de diez ejecuciones")
plt.grid(axis="y", alpha=0.3)
plt.show()
```

```
Método           Mínimo      Media      Desv.
Aleatoria        3432.0     3707.4      155.6
Local            1315.0     1358.5       30.3
Recocido         1273.0     1370.8       46.0
```



La búsqueda aleatoria presenta las distancias más altas porque cada iteración comienza desde cero. La búsqueda local mejora de forma consistente, aunque puede detenerse en distintos mínimos locales. El recocido simulado acepta temporalmente soluciones peores y obtiene los mejores resultados del experimento con los parámetros utilizados.